

NOTICE

- Deadlines
 - (5/13 11:59 PM) Programming assignment 3
 - (5/15 11:59 PM) Midterm Quiz 3

CS 374: OPERATING SYSTEMS I

PART III – NETWORKING II

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI

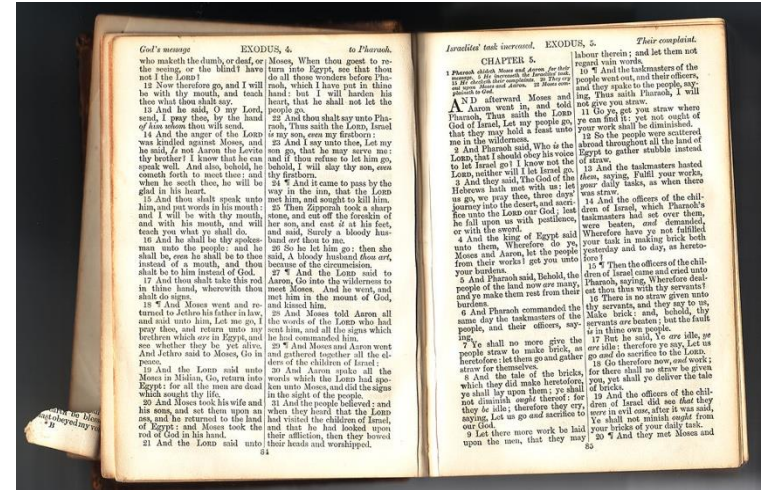
TOPICS FOR TODAY

- Part III: Networking
 - OTP
 - What is one-time pad?
 - How does it work?
 - OTP in the real-world
 - OTP in programming assignment IV
 - HTTPS and Man-in-the-Middle
 - What is HTTP and why is this a problem?
 - What is HTTP+S?
 - What TLS does and how it works?
 - Man-in-the-Middle (MitM) attack
 - MitM attack instantiations and defenses

MEDIEVAL CIPHER

• Example

- Password: _ _ _ _ (4-digit numbers)
- Hint:
 - “keep on loving each other as brothers”
 - “fear not, for I am with you”
 - “You will not certainly die,” the serpent said
 - “Behold, I have told you before”



MEDIEVAL CIPHER

• Example

- Password: _ _ _ _ (4-digit numbers)
- Hint:
 - “keep on loving each other as brothers”
 - “fear not, for I am with you”
 - “You will not certainly die,” the serpent said
 - “Behold, I have told you before”
- Solution: 4 2 5 0
 - “keep ... ” > Hebrews > 13: **4**
 - “fear n...” > Isaiah > 66: **2**
 - “You w...” > Genesis > 50: **5**
 - “Behol...” > Matthew > 28: **0**



MEDIEVAL CIPHER

- Potential problems

- Key is not completely random
- OTP is known to the attacker
- Plaintext and its ciphertext are known
- OTP is used incorrectly by human

Brute-force

Done (e.g., King James Version)

Key recovery

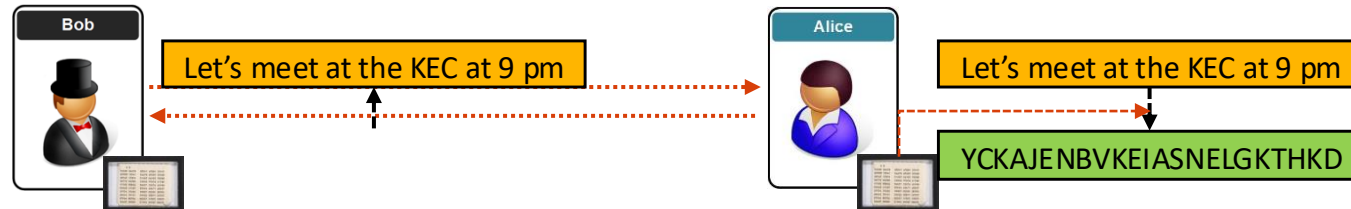
Plaintext recovery

ONE-TIME PAD

- What is it?
 - One-Time Pads (OTP) is an encryption mechanism
- How it works?
 - Alice and Bob want to communicate *securely*
 - Alice and Bob share the same OTP
 - Alice encrypts a message to send with the OTP
 - Alice sends the encrypted message to Bob
 - Bob decrypts the received message with the OTP



An Example OTP



ONE-TIME PAD: ENCRYPTION

- Encryption example

- Taken from Wikipedia ([link](#))
- Alice wants to say “hello” to Bob
(Key chosen from OTP: XMCKL)

	h	e	l	l	o	message
	7 (h)	4 (e)	11 (l)	11 (l)	14 (o)	message
+	23 (X)	12 (M)	2 (C)	10 (K)	11 (L)	key
=	30	16	13	21	25	message + key
=	4 (E)	16 (Q)	13 (N)	21 (V)	25 (Z)	(message + key) mod 26
	E	Q	N	V	Z	→ ciphertext

- Alice’s “hello” becomes “EQNVZ”
- Alice sends “EQNVZ” to Bob
- **Enc(m, k) := [(m + k) mod 26]**

EQNVZ: Ciphertext (the output of an encryption)
hello : Plaintext (the text we want to encrypt)
XMCKL: Key (the text we use for the encryp-/decryption)

ONE-TIME PAD: DECRYPTION

- **Decryption example**

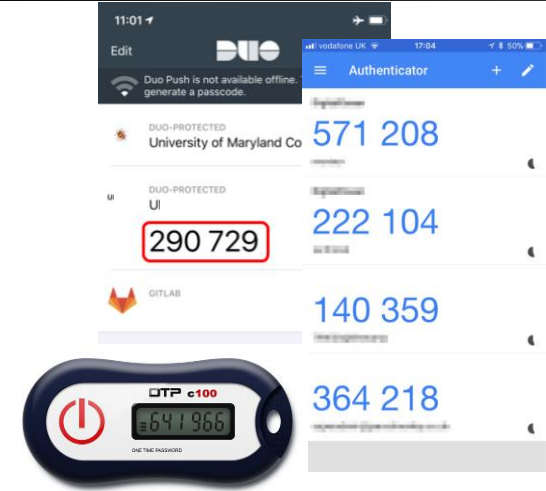
- Bob receives “EQNVZ” from Alice
- Bob has the same key chosen from OTP (XMCKL)

	E		Q		N		V		Z	ciphertext
	4 (E)		16 (Q)		13 (N)		21 (V)		25 (Z)	ciphertext
–	23 (X)		12 (M)		2 (C)		10 (K)		11 (L)	key
=	-19		4		11		11		14	ciphertext – key
=	7 (h)		4 (e)		11 (l)		11 (l)		14 (o)	ciphertext – key (mod 26)
	h		e		l		l		o	→ message

- Alice’s “EQNVZ” now becomes “hello”
- **$\text{Dec}(c, k) := [(c - k) \bmod 26]$**

DO NOT CONFUSE WITH

- **One-Time Password (OTP)**
 - One-Time Password is a *password* only valid for one session



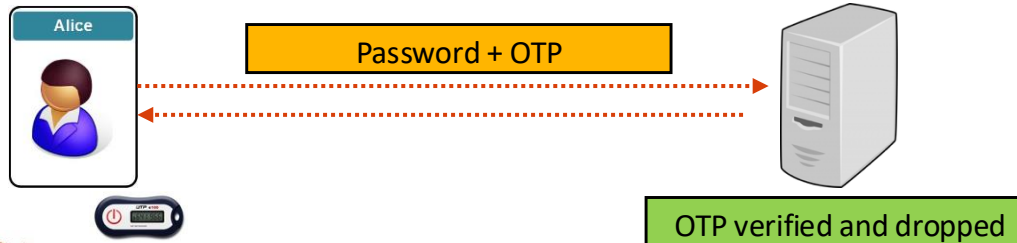
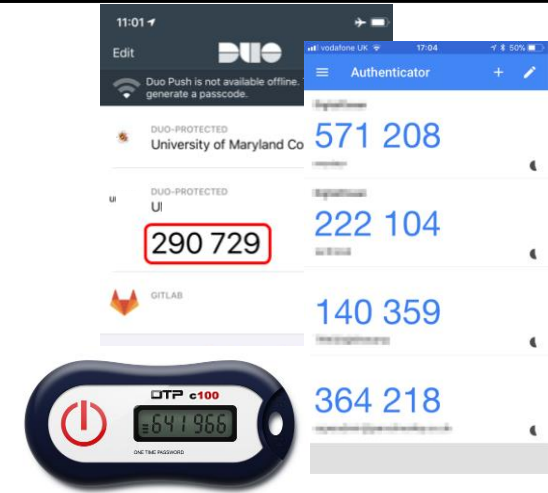
DO NOT CONFUSE WITH

- **One-Time Password (OTP)**

- One-Time Password is a *password* only valid for one session

- **How it works?**

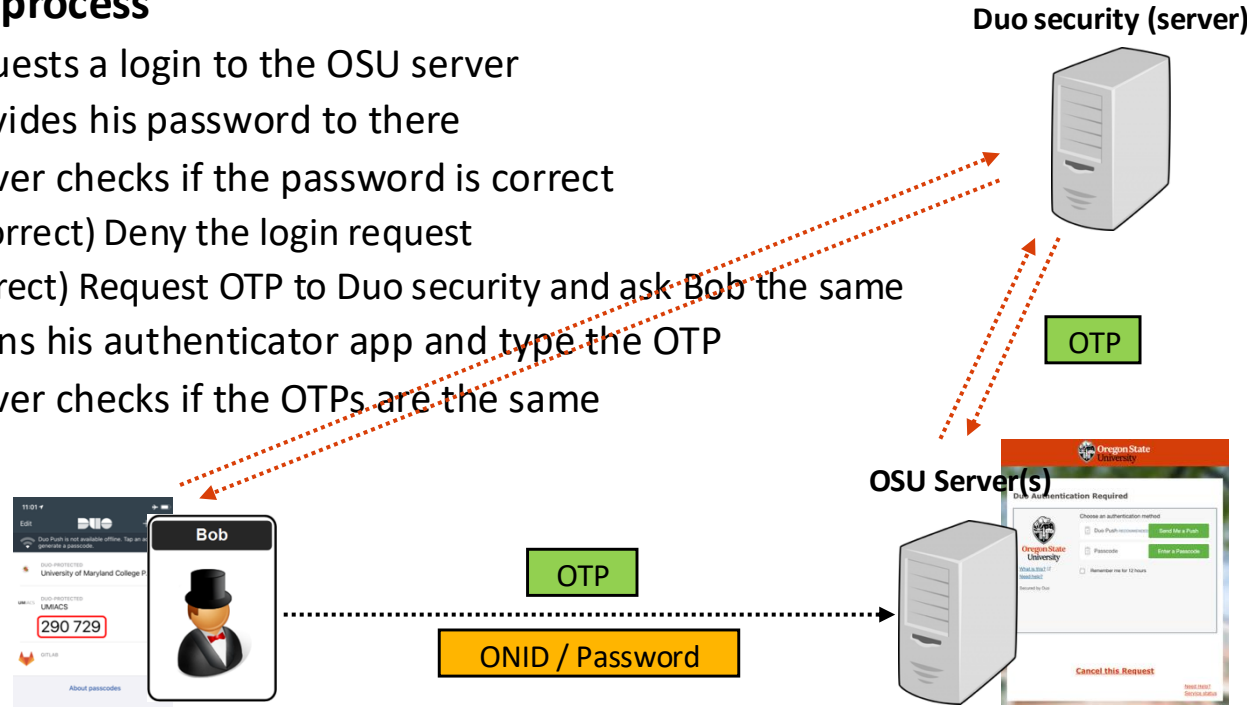
- Alice logs in
- Alice enters her password plus a one-time password
- The server verifies whether the OTP is valid for that session
- The OTP cannot be reused



ONE-TIME PASSWORD

- **OSU log-in process**

- Bob requests a login to the OSU server
- Bob provides his password to there
- OSU server checks if the password is correct
 - (Incorrect) Deny the login request
 - (Correct) Request OTP to Duo security and ask Bob the same
- Bob opens his authenticator app and type the OTP
- OSU server checks if the OTPs are the same



ONE-TIME PAD: PROGRAMMING ASSIGNMENT IV

- **Required programs**

- (keygen) Key generator
- (enc_server) Encryption server
- (enc_client) Encryption client
- (dec_server) Decryption server
- (dec_client) Decryption client

ONE-TIME PAD: PROGRAMMING ASSIGNMENT IV

- **Overall process**

- (keygen) Alice generates a key via a keygen program
- (keygen) Bob has the *same* key (do not re-generate)
- Suppose there are two servers
 - (enc_server) Encrypt a plaintext using a key
 - (dec_server) Decrypt a ciphertext using a key

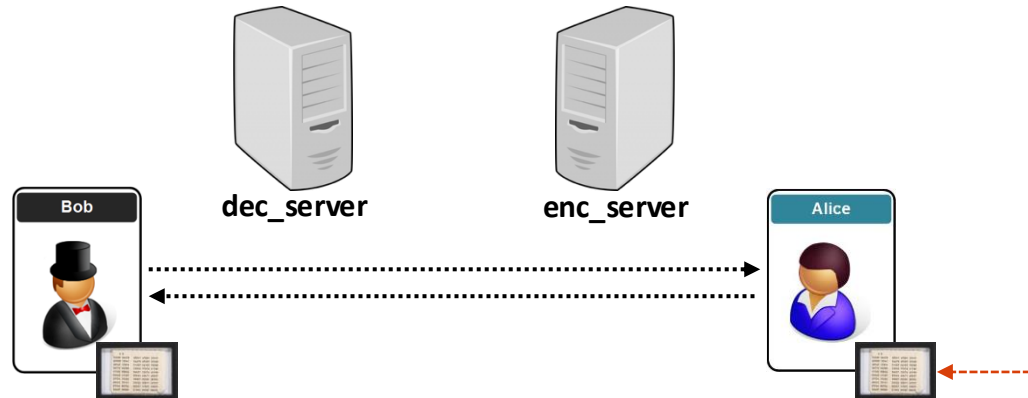
(keygen)

Input: key length (int)

Output: a *randomly* generated key

ex. \$./keygen 10

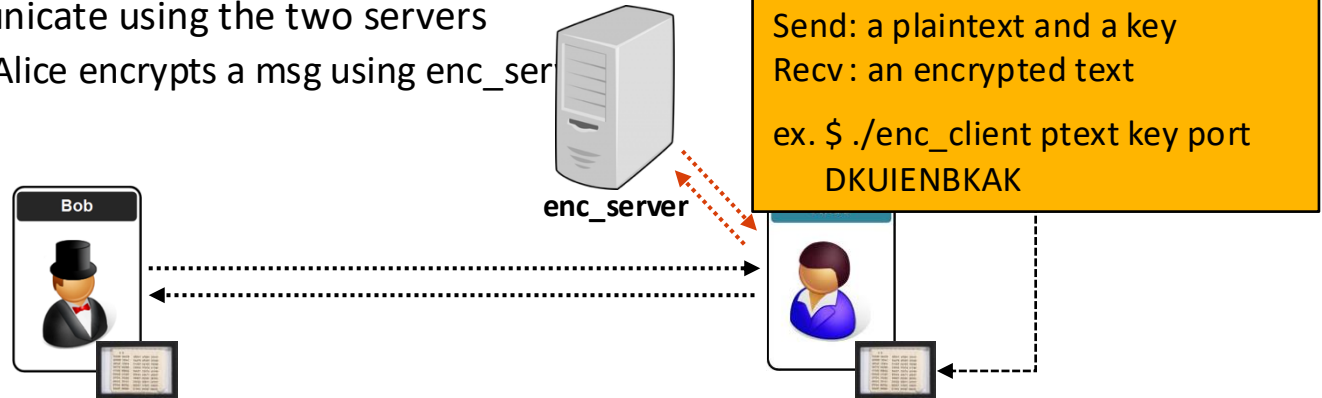
QKASLKNGDK



ONE-TIME PAD: PROGRAMMING ASSIGNMENT IV

- **Overall process**

- (keygen) Alice generates a key via a keygen program
- (keygen) Bob has the *same* key (do not re-generate)
- Suppose there are two servers
 - (enc_server) Encrypt a plaintext using a key
 - (dec_server) Decrypt a ciphertext using a key
- Securely communicate using the two servers
 - (enc_client) Alice encrypts a msg using enc_ser



ONE-TIME PAD: PROGRAMMING ASSIGNMENT IV

• Overall process

- (keygen) Alice generates a key via a keygen program
- (keygen) Bob has the *same* key (do not re-generate)
- Suppose there are two servers
 - (enc_server) Encrypt a plaintext using a key
 - (dec_server) Decrypt a ciphertext using a key
- Securely communicate using the two servers
 - (enc_client) Alice encrypts a plaintext using enc_server
 - (dec_client) Bob decrypts the ciphertext using dec_server

(keygen)

Input: key length (int)

Output: a *randomly* generated key

ex. `$ ./keygen 10`

QKASLKNGDK

(enc_client > enc_server)

Send: a plaintext and a key

Recv: an encrypted text

ex. `$ ./enc_client ptext key port`

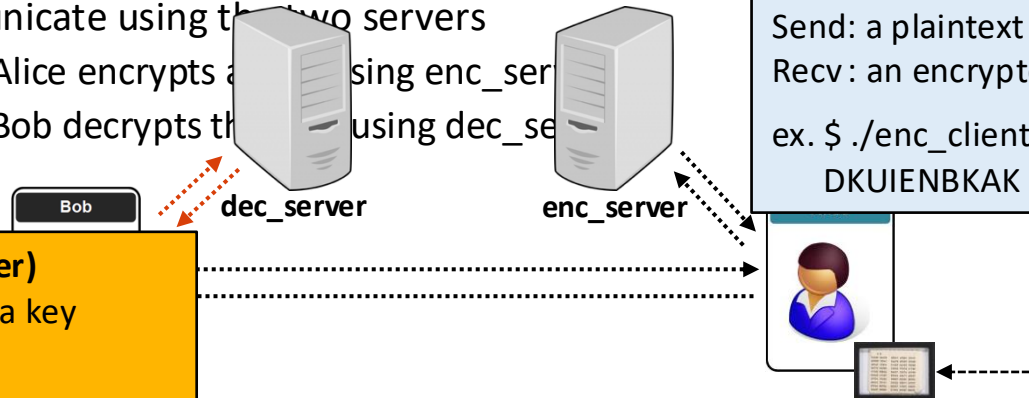
DKUIENBKAK

(dec_client > dec_server)

Send: a ciphertext and a key

Recv: a plaintext

ex. `$ ./dec_client ctext key port`
ptext



REVISIT NETWORKING: PORT

- Port
 - **Formal:** A communication endpoint (defined at the transport layer)
 - **TL; DR :** A number (0 – 65535) that must be associated with an IP for communication
- Notation
 - <IP address>:<Port number>
 - ex. 76.298.83.129:433
 - IP address: 76.298.83.129 | Port #: 443
- Ports reserved in Linux
 - 22: SSH connection
 - 80: HTTP
 - 443: HTTPS
 - 2967: Symantec AV
 - 6112: Battle.net

Tip:

- Use port # in the 50000+ range
- Use different port number every time you run the server
[Note: it is still unavailable for some time after your program terminates]
- Oftentimes, a port is already used by your fellow; then choose others
- \$ netstat -tulp | grep LISTEN (to see used ports)

ONE-TIME PAD: CLIENT-SERVER ARCHITECTURE

- **OTP in PA V**

- (keygen) Alice generates a key via a keygen program
- (keygen) Bob has the *same* key (do not re-generate)
- Suppose there are two servers
 - (enc_server) Encrypt a plaintext using a key
 - (dec_server) Decrypt a ciphertext using a key
- Securely communicate using the two servers
 - (enc_client) Alice encrypts a plaintext using enc_server
 - (dec_client) Bob decrypts the ciphertext using dec_server

(keygen)

Input: key length (int)

Output: a *randomly* generated key

ex. \$./keygen 10

QKASLKN GDK

(enc_client > enc_server)

Send: a plaintext and a key

Recv: an encrypted text

ex. \$./enc_client ptext key port

DKUIENBKAK

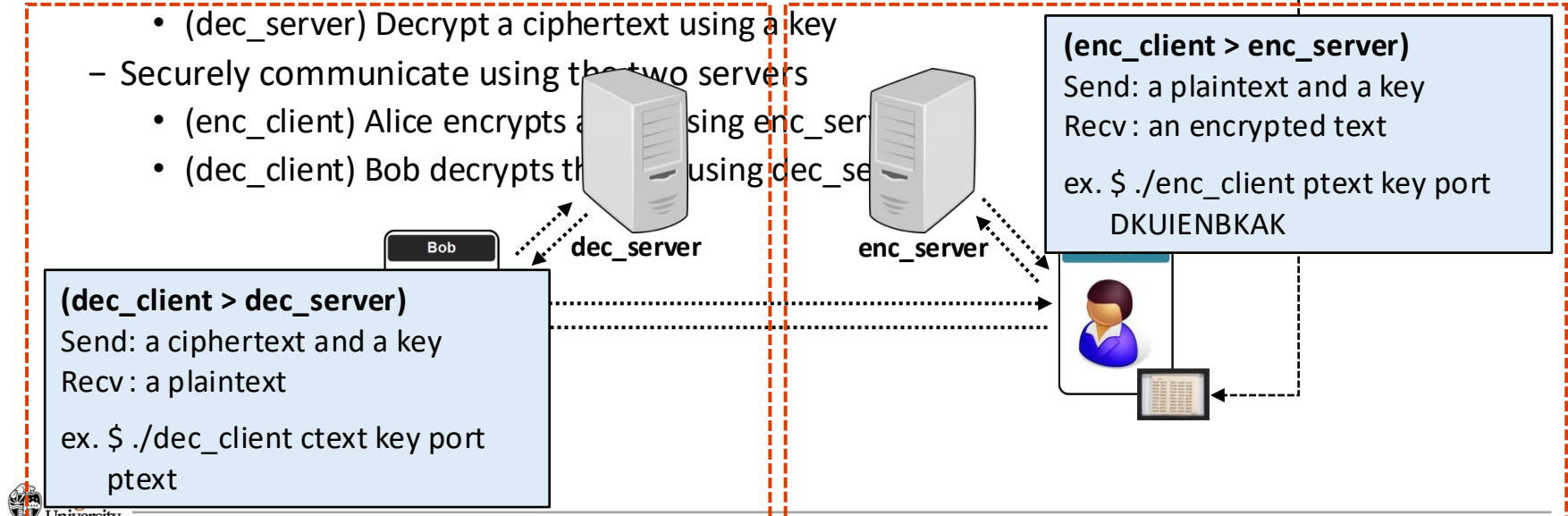
(dec_client > dec_server)

Send: a ciphertext and a key

Recv: a plaintext

ex. \$./dec_client ctext key port

ptext



REVISIT: CLIENT-SERVER PROGRAMMING (CLIENT.C)

```
#define IPADDR <SERVER_IP>
#define PORT <SERVER_PORT>
#define BUFSIZE 1024
```

```
AF_INET (IPv4)
SOCK_STREAM (bi-directional)
```

```
int main(int argc, char *argv)
{
    int sock = 0, valread;
    struct sockaddr_in serv_addr;
    char* hello = "Hello (client)";
    char buffer[BUFSIZE] = { 0 };
```

```
// create a socket
if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    printf("Error: socket creation error\n");
    return -1;
}
```

```
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);
```

```
// convert IP addresses from text to binary
if (inet_pton(AF_INET, IPADDR, &serv_addr.sin_addr) <= 0) {
    printf("Error: invalid address, address not supported\n");
    return -1;
}
```

```
if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Connection Failed\n");
    return -1;
}
```

```
// request encryption/decryption
send(sock, hello, strlen(hello), 0);
printf("Message sent (client)\n");
valread = recv(sock, buffer, BUFSIZE);
printf("%s\n", buffer);
```

```
return 0;
}
```

Connect to the server, running on the IP address we specify "127.0.0.1"

Our OTP case:

1. Send a plan/ciphertext and a key
2. Receive a cipher/plaintext

REVISIT: CLIENT-SERVER PROGRAMMING (SERVER.C)

Bind the socket to the address
> Any IP (of the host)
> Port # 8080

... omit the includes

```
#define BUF_SIZE 1024
#define PORT     SERVER_PORT
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";
```

```
// create socket (returns a file descriptor for read/write
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed"); exit(EXIT_FAILURE);
}
```

```
// (you can skip) attach this socket to the port number 8080
if (setsockopt(server_fd, SOL_SOCKET,
    SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
    perror("setsockopt failed"); exit(EXIT_FAILURE);
}
```

```
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY; // bind to any address
address.sin_port = htons(PORT);       // format the port num
```

AF_INET (IPv4)
SOCK_STREAM (bi-directional)

SO_REUSEADDR
SO_REUSEPORT
opt (optional value)

```
// attach socket to the port 8080
if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
    perror("bind failed"); exit(EXIT_FAILURE);
}
```

```
if (listen(server_fd, 3) < 0) {
    perror("listen failed"); exit(EXIT_FAILURE);
}
```

```
while (1) {
    if ((new_socket = accept(server_fd,
        (struct sockaddr*)&address,
        (socklen_t*)&sizeof(address))) < 0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }
```

```
    valread = read(new_socket, buffer, 1024);
    printf("%s\n", buffer);
    send(new_socket, hello, strlen(hello), 0);
    printf("Message sent (server)\n");
    close(new_socket);
}
close(server_fd);
return 0;
}
```

Listen incoming connections
> Use the socket fd
> Allow 3 connections (max.)

Our OTP case:

1. Receive a plan/ciphertext and a key
2. Send a cipher/plain text

TOPICS FOR TODAY

- Part III: Networking
 - OTP
 - What is one-time pad?
 - How does it work?
 - OTP in the real-world
 - OTP in programming assignment IV
 - HTTPS and Man-in-the-Middle
 - What is HTTP and why is this a problem?
 - What is HTTP+S?
 - What TLS does and how it works?
 - Man-in-the-Middle (MitM) attack
 - MitM attack instantiations and defenses

HTTP

- What is it?

- HyperText Transfer Protocol
- Operates at the application layer (OSI Layer 7)

- Example

- \$ nc -l 8080
- \$ sudo tcpdump -A -i lo0 port 8080
- \$./http

... many includes

```
int main(void) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port   = htons(8080),
    };
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
    connect(sock, (struct sockaddr *)&addr, sizeof(addr));

    char *req =
        "POST /login HTTP/1.1\r\n"
        "Host: example.com\r\n"
        "Content-Type: application/x-www-form-urlencoded\r\n"
        "Content-Length: 27\r\n"
        "\r\n"
        "username=alice&password=1234";
    send(sock, req, strlen(req), 0);

    char buf[1024] = {0};
    recv(sock, buf, sizeof(buf), 0);
    printf("%s\n", buf);

    close(sock);
    return 0;
}
```

HTTP

- What is it?

- HyperText Transfer Protocol
- Operates at the application layer (OSI Layer 7)

- Problems

- Confidentiality
 - HTTP sends all data as *plaintext* over the network
 - Any router, proxy, or device on the path can read it
 - Credentials, cookies, and form data are visible
- Integrity
 - Data can be silently altered in transit

... many includes

```
int main(void) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);

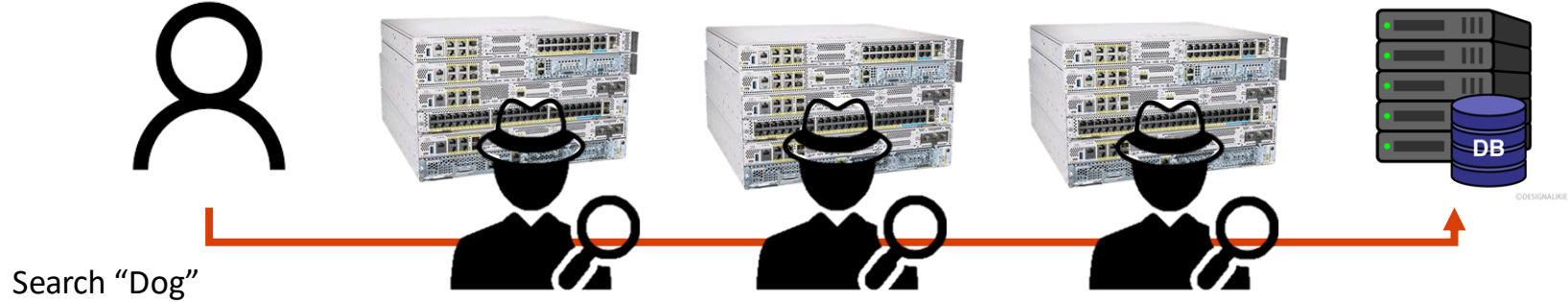
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(8080),
    };
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
    connect(sock, (struct sockaddr *)&addr, sizeof(addr));

    char *req =
        "POST /login HTTP/1.1\r\n"
        "Host: example.com\r\n"
        "Content-Type: application/x-www-form-urlencoded\r\n"
        "Content-Length: 27\r\n"
        "\r\n"
        "username=alice&password=1234";
    send(sock, req, strlen(req), 0);

    char buf[1024] = {0};
    recv(sock, buf, sizeof(buf), 0);
    printf("%s\n", buf);

    close(sock);
    return 0;
}
```

NETWORK COMM. WITHOUT SECURITY



**Everybody in the Middle Knows That I Searched 'dogs'
and They Also Know the Search Result... Ugh...**



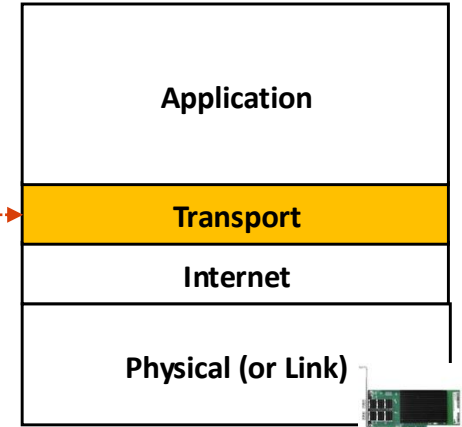
HTTPS

- **What is HTTPS?**

- HTTP Secure: HTTP running on top of TLS
- TLS: Transport Layer Security
- Default port: 443 (HTTP uses port 80)

- **TLS Provides**

- Confidentiality : data is encrypted in transit
- Integrity : data cannot be silently modified
- Authentication : we know who we talk to



HTTPS

- **(Simplified) TLS Handshake Process**

- Client hello : with cipher suites and a random number
- Server hello : with a chosen cipher suites and its TLS certificate
- Client verifies cert : signed by a trusted certificate authority (CA)
- Session key exchange : both client and server derive a shared symmetric session key
- Encrypted comm. starts : with the session key used to encrypt data

Middle Trans Never Know

Check certificate, exchange keys, early cooperation with UMAG



```
0x1ce42780dfa1cea  
089a9ea00de059ef5
```

I know these two are communicating but not about the secret key...



This micrograph shows a dark, granular polymer matrix with numerous small, bright, needle-like or rod-like structures dispersed throughout. These structures are carbon nanotubes, which appear as high-contrast features against the darker background of the polymer.

**The Middlemen Will Only See the Encrypted Contents
They Will **Never** Know the Secret Key ...**

TOPICS FOR TODAY

- Part III: Networking
 - OTP
 - What is one-time pad?
 - How does it work?
 - OTP in the real-world
 - OTP in programming assignment IV
 - HTTPS and Man-in-the-Middle
 - What is HTTP and why is this a problem?
 - What is HTTP+S?
 - What TLS does and how it works?
 - Man-in-the-Middle (MitM) attack
 - MitM attack instantiations and defenses

MAN-IN-THE-MIDDLE ATTACK



- **MitM scenario**
 - You connect to “Starbucks WiFi”
 - But anyone can sit between you and the attacker



MAN-IN-THE-MIDDLE ATTACK



- **MitM scenario**

- You connect to “Starbucks WiFi”
- But anyone can sit between you and the attacker
- The attacker can
 - read plaintext credentials, session tokens, private data
 - alter data in transit (inject malicious content)

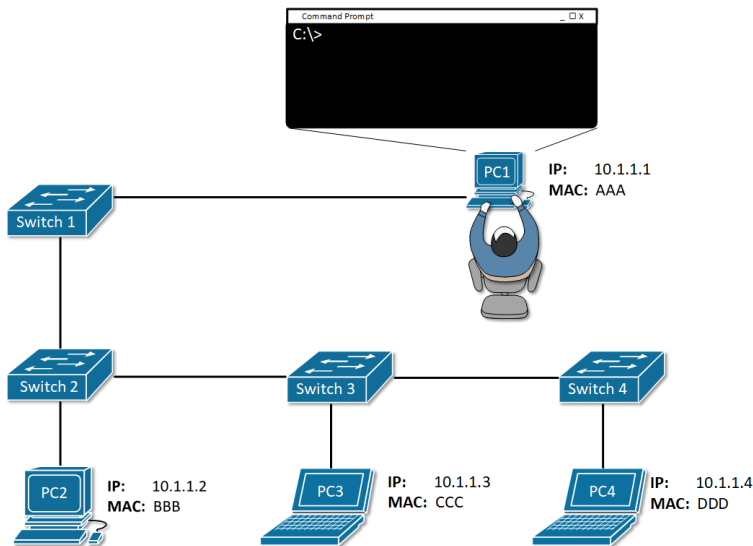
MAN-IN-THE-MIDDLE ATTACK

- **What is a MitM attack?**
 - Attacker secretly intercepts communication between two parties
 - The two parties are *not* aware of the attacker's presence
- **What can the attacker do?**
 - Read plaintext credentials, session tokens, private data
 - Alter data in transit (inject malicious content)

MAN-IN-THE-MIDDLE ATTACK: ARP SPOOFING

- What is ARP?

- **Definition:** ARP (address resolution protocol) that maps an IP address to MAC address
- Used on a local network at data-link layer (OSP Layer 2)
- ARP is *stateless*: host accept replies without first sending a request



Dst is well-known broadcast addr.
Target MAC: filled with 0, means the IP owner asks this IP addr.

Src is filled with the MAC addr, associated with the requested IP

ARP Request

Ethernet II Frame

Src: AAAA-AAAA-AAAA

Dst: FFFF-FFFF-FFFF

Address Resolution Protocol (request)

Sender MAC: AAAA-AAAA-AAAA

Sender IP: 10.1.1.1

Target MAC: 0000-0000-0000

Target IP: 10.1.1.3

ARP Reply

Ethernet II Frame

Src: CCCC-CCCC-CCCC

Dst: AAAA-AAAA-AAAA

Address Resolution Protocol (request)

Sender MAC: CCCC-CCCC-CCCC

Sender IP: 10.1.1.3

Target MAC: AAAA-AAAA-AAAA

Target IP: 10.1.1.1

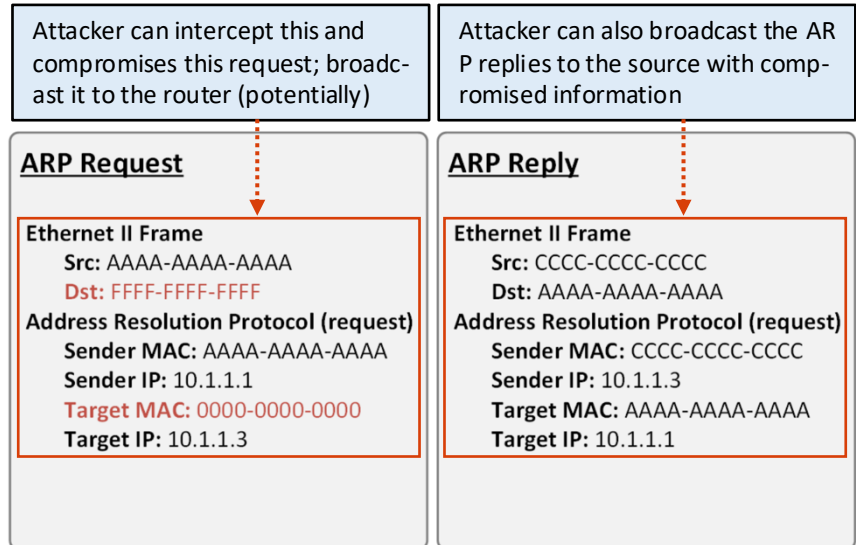
MAN-IN-THE-MIDDLE ATTACK: ARP SPOOFING

- What is ARP?

- **Definition:** ARP (address resolution protocol) that maps an IP address to MAC address
- Used on a local network at data-link layer (OSP Layer 2)
- ARP is *stateless*: host accept replies without first sending a request

- ARP Spoofing attack

- Attacker *broadcasts fake ARP replies*
- Alice's ARP cache poisoned
- Gateway's ARP cache poisoned
- Alice's traffic flows through the attacker
- Detection: `$ arp -a / $ arp-scan --localnet`



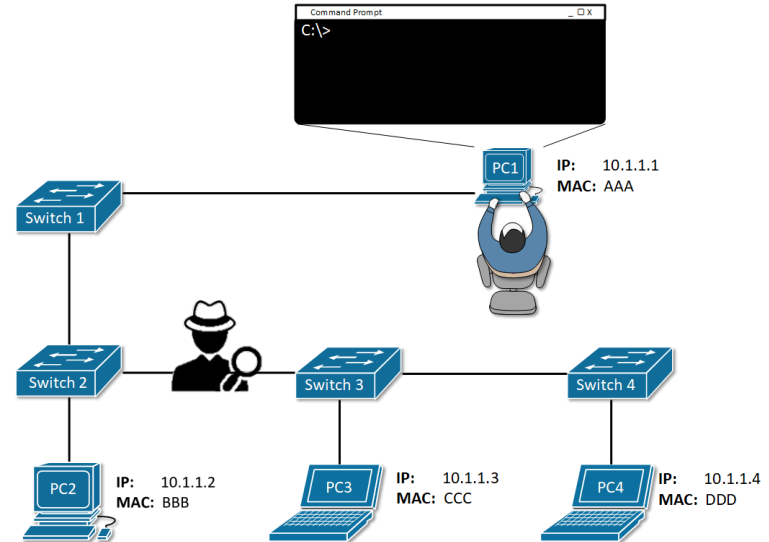
MAN-IN-THE-MIDDLE ATTACK: ARP SPOOFING

- What is ARP?

- **Definition:** ARP (address resolution protocol) that maps an IP address to MAC address
- Used on a local network at data-link layer (OSP Layer 2)
- ARP is *stateless*: host accept replies without first sending a request

- ARP Spoofing attack

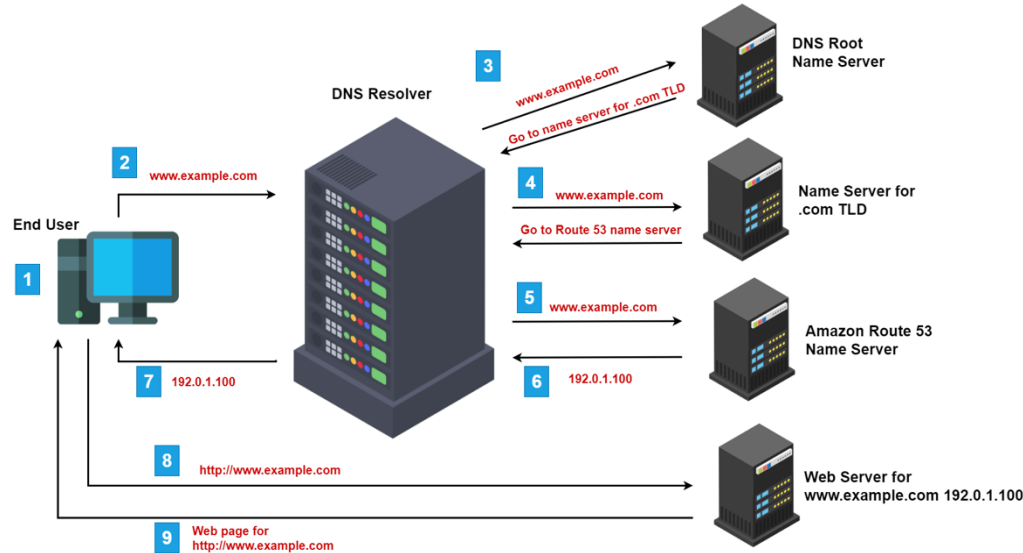
- Attacker *broadcasts fake ARP replies*
- Alice's ARP cache poisoned
- Gateway's ARP cache poisoned
- Alice's traffic flows through the attacker
- Detection: `$ arp -a` / `$ arp-scan --localnet`



MAN-IN-THE-MIDDLE ATTACK: DNS SPOOFING

- What is DNS?

- **Definition:** DNS (domain name system) that maps URLs to IP addresses
- DNS requests go out in plaintext on UDP port 53
- Your laptop asks the local DNS resolver – which the attacker can control



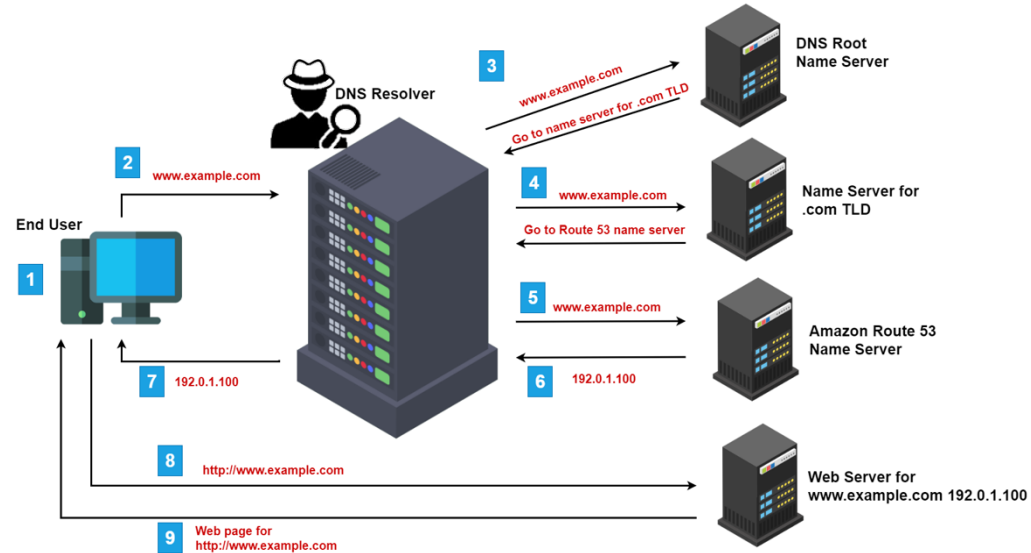
MAN-IN-THE-MIDDLE ATTACK: DNS SPOOFING

- What is DNS?

- **Definition:** DNS (domain name system) that maps URLs to IP addresses
- DNS requests go out in plaintext on UDP port 53
- Your laptop asks the local DNS resolver – which the attacker can control

- DNS spoofing attack

- Intercepts your DNS request (https://bank.com)
- Replies with their IP: 2.3.5.6
- Your browser connects to the fake bank web page
- Page looks identical, and you type your credentials
- \$ nslookup bank.com



Thank You!

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI